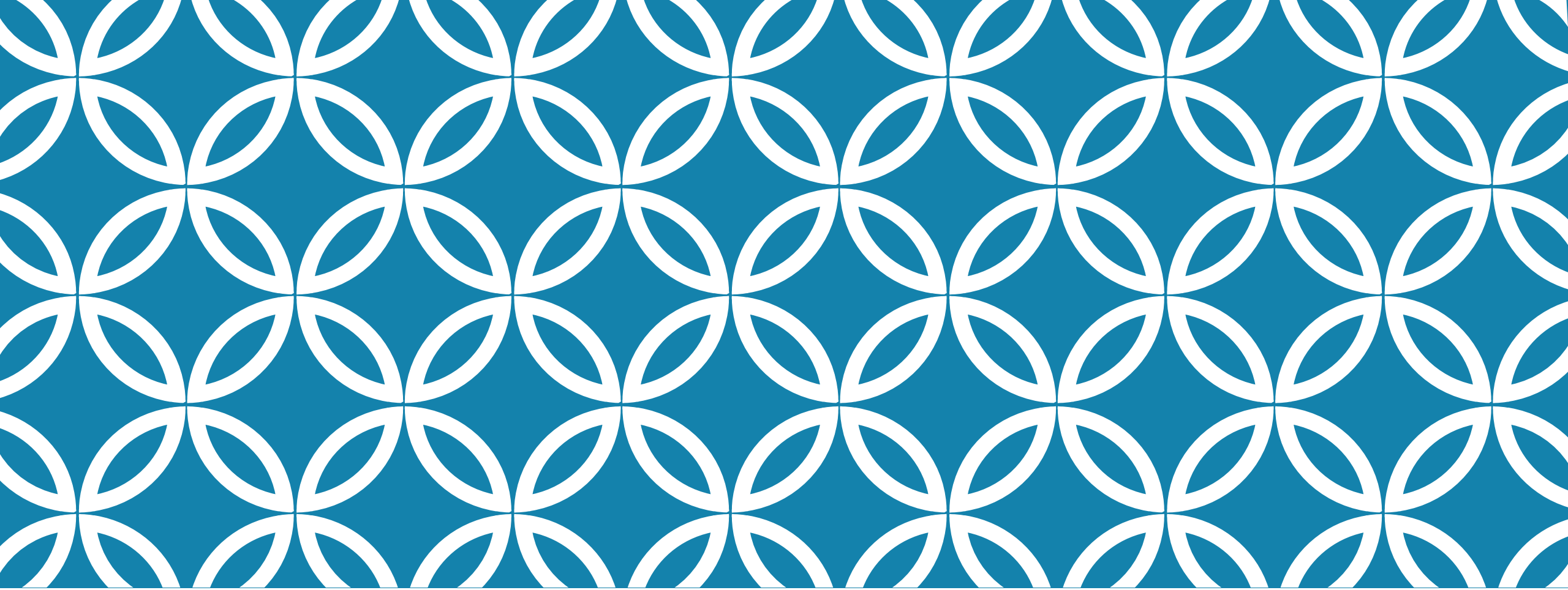


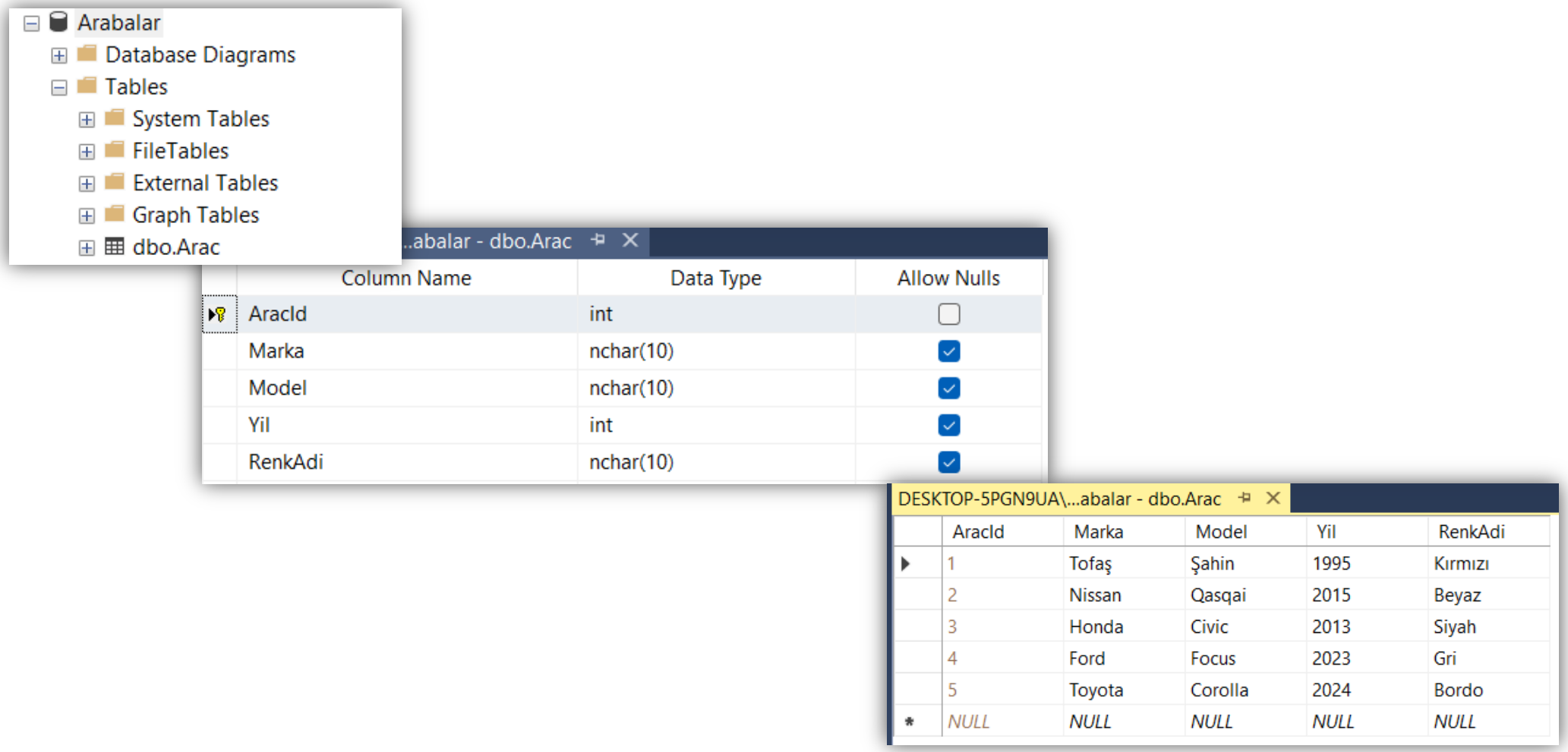
GÖRSEL PROGRAMLAMA-II

Öğr. Gör. Erhan AKAGÜNDÜZ

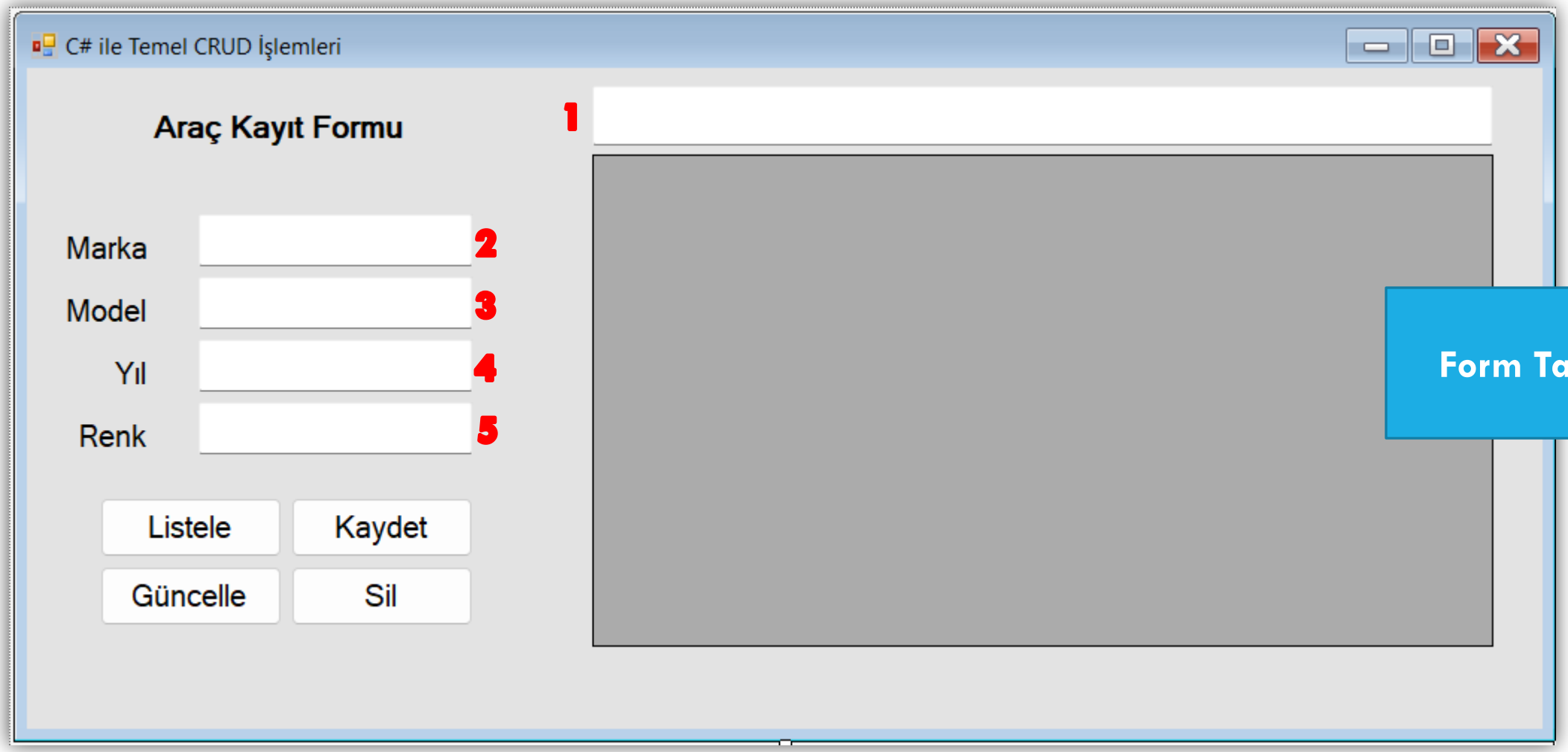
C# İLE TEMEL CRUD İŞLEMLERİ

- **FORM TASARIMI**
- Visual Studio 2022 Community ile bir firmaya ait araçların bilgilerinin tutulduğu bir uygulamaya yapacağız.
- Bunun için öncelikle MS SQL Server da Veritabanı ve Tablo oluşturulacaktır.
- Daha sonra Visual Studio 2022 de form tasarımı yapılacaktır.
- En son olarak da gerekli işlemler için ilgili kodlamalar yapılacak ve program hazır hale getirilecektir.

C# İLE TEMEL CRUD İŞLEMLERİ



C# İLE TEMEL CRUD İŞLEMLERİ



C# İLE TEMEL CRUD İŞLEMLERİ

No	Kontrol Adı (Component)	Özellik (Property)	Değer (Value)
1	TextBox	Name	txtAra
2	TextBox	Name	txtMarka
3	TextBox	Name	txtMode
4	TextBox	Name	txtYil
5	TextBox	Name	txtRenk
6	Button	Name Text	btnKaydet Kaydet
7	Button	Name Text	btnListele Listele
8	Button	Name Text	btnGuncelle Güncelle
9	Button	Name Text	btnSil Sil
10	DataGridView	Name	dataGridArac

C# İLE TEMEL CRUD İŞLEMLERİ

- **KAYIT EKLEME**
- ADO.NET ile veri tabanı üzerinde bulunan bir tabloya kayıt eklemek için **SqlCommand** nesnesinin **“CommandText”** özelliğine ilgili **“Insert Into”** deyimi tanımlandıktan sonra **ExecuteNonQuery()** metodu çağırılır.
- **“btnKaydet”** kontrolünün **“Click”** olayı:

C# İLE TEMEL CRUD İŞLEMLERİ

```
public Form1()
{
    InitializeComponent();
}

public string conString = "Data Source=DESKTOP-5PGN9UA\\SQLEXPRESS; Initial Catalog=Arabalar; Integrated
  Security=True;";

1 başvuru
private void btnKaydet_Click(object sender, EventArgs e)
{
    // SqlConnection ile veritabanına bağlantı oluşturuluyor; using ifadesi ile otomatik kapama
      sağlanıyor.
    using (SqlConnection connect = new SqlConnection(conString))
    {
        try
        {
            connect.Open(); // Bağlantıyı aç

            // Arac tablosuna veri eklemek için SQL INSERT komutu tanımlanıyor.
            string kayit = "INSERT INTO Arac (Marka, Model, Yil, RenkAdi) VALUES (@marka, @model, @yil,
              @renkadi)";
```

C# İLE TEMEL CRUD İŞLEMLERİ

```
        // SqlCommand nesnesi ile SQL komutu ve bağlantı ilişkilendiriliyor.
        using (SqlCommand komut = new SqlCommand(kayit, connect))
        {
            // Parametreler ekleniyor; kullanıcıdan alınan veriler SQL sorgusuna dahil ediliyor.
            komut.Parameters.Add(new SqlParameter("@marka", txtMarka.Text));
            komut.Parameters.Add(new SqlParameter("@model", txtModel.Text));
            komut.Parameters.Add(new SqlParameter("@yil", int.Parse(txtYil.Text))); // int türüne
              dönüşüm
            komut.Parameters.Add(new SqlParameter("@renkadi", txtRenk.Text));

            // SQL komutu çalıştırılarak veri ekleniyor.
            komut.ExecuteNonQuery();
        }

        // Kayıt işlemi başarılıysa kullanıcıya bilgi mesajı gösteriliyor.
        MessageBox.Show("Yeni Kayıt Eklendi", "Mesaj Kutusu", MessageBoxButtons.OKCancel,
          MessageBoxIcon.Information);
    }
    catch (Exception hata)
    {
        // Hata oluşursa kullanıcıya hata mesajı gösteriliyor.
        MessageBox.Show("Hata Meydana Geldi: " + hata.Message, "Mesaj Kutusu",
          MessageBoxButtons.OKCancel, MessageBoxIcon.Error);
    }
    // using bloğu sayesinde bağlantı otomatik olarak kapatılır.
}
```

C# İLE TEMEL CRUD İŞLEMLERİ

- **KAYIT LİSTELEME**
- ADO.NET ile veri tabanında bulunan bir tablodan kayıt okumak için SqlDataReader veya SqlDataAdapter yöntemleri tercih edilebilir.
- Eğer kayıtlar bir **DataGridView** içerisinde listelenecekse **SqlDataAdapter** kullanımı daha uygun bir tercih olacaktır.
- DataGridView'ın özellikler penceresinde, tablo boyutuna göre otomatik boyutlandırma için **AutoSizeColumnsMode** özelliğini **Fill** yapılabilir.
- **"btnListele"** kontrolünün **"Click"** olayı:

C# İLE TEMEL CRUD İŞLEMLERİ

```
private void btnListele_Click(object sender, EventArgs e)
{
    SqlConnection sqlConnection = new SqlConnection(conString);
    try
    {
        sqlConnection.Open(); // Bağlantıyı aç

        // Arac tablosundaki tüm kayıtları almak için SQL komutu oluşturuluyor.
        SqlCommand komut = new SqlCommand("Select * From Arac", sqlConnection);

        // SQL komutunu kullanarak verileri almak için SqlDataAdapter oluşturuluyor.
        SqlDataAdapter sqlDataAdapter = new SqlDataAdapter(komut);

        // Verileri depolamak için yeni bir DataSet oluşturuluyor.
        DataSet dataSet = new DataSet();

        // SqlDataAdapter ile DataSet'e veriler dolduruluyor.
        sqlDataAdapter.Fill(dataSet);

        // DataGrid'e veri kaynağı olarak DataSet'teki ilk tablo atanıyor.
        dataGridArac.DataSource = dataSet.Tables[0];
```

C# İLE TEMEL CRUD İŞLEMLERİ

```
        // Sütun isimlerini değiştirme
        dataGridArac.Columns[0].HeaderText = "Araç ID"; // AracId
        dataGridArac.Columns[1].HeaderText = "Marka"; // Marka
        dataGridArac.Columns[2].HeaderText = "Model"; // Model
        dataGridArac.Columns[3].HeaderText = "Yıl"; // Yil
        dataGridArac.Columns[4].HeaderText = "Renk"; // Renk

    }
    catch (Exception hata)
    {
        // Hata oluşursa kullanıcıya bir mesaj gösteriliyor.
        MessageBox.Show("Hata Meydana Geldi: " + hata.Message, "Mesaj Kutusu", MessageBoxButtons.OKCancel,
            MessageBoxIcon.Error);
    }
    finally
    {
        // Bağlantı kapatılıyor.
        sqlConnection.Close();
    }

}
```

C# İLE TEMEL CRUD İŞLEMLERİ

- **KAYIT BULMA**
- Veri tabanında bulunan bir tablodan sadece belirlenen koşullara uyan kayıtları filtrelemek için "**Select**" ifadesi "**Where**" deyimi ile birlikte kullanılmaktadır.
- **"txtAra"** kontrolünün **"TextChanged"** olayı:

C# İLE TEMEL CRUD İŞLEMLERİ

```
private void txtAra_TextChanged(object sender, EventArgs e)
{
    // SqlConnection ile veritabanına bağlantı oluşturuluyor; using ifadesi ile otomatik kapama
      sağlanıyor.
    using (SqlConnection sqlConnection = new SqlConnection(conString))
    {
        try
        {
            sqlConnection.Open(); // Bağlantıyı aç

            // Arac tablosundan arama yapacak SQL komutu tanımlanıyor.
            SqlCommand sqlCommand = new SqlCommand();
            sqlCommand.Connection = sqlConnection;

            // LIKE ifadesi ile arama yapılıyor. Aranacak ifade için wildcards kullanılıyor.
            sqlCommand.CommandText = @"SELECT AracId, Marka, Model, Yil, RenkAdi FROM Arac WHERE Marka
             LIKE '%' + @AranacakIfade +
             '%' OR Model LIKE '%' + @AranacakIfade +
             '%' OR CAST(Yil AS nvarchar(4)) LIKE '%'
             + @AranacakIfade + '%' OR RenkAdi LIKE '%' + @AranacakIfade + '%'";

            // Kullanıcıdan alınan arama ifadesi parametre olarak ekleniyor.
            sqlCommand.Parameters.AddWithValue("@AranacakIfade", txtAra.Text);
```

C# İLE TEMEL CRUD İŞLEMLERİ

```
            // SqlDataAdapter ile verileri almak için oluşturuluyor.
            SqlDataAdapter sqlDataAdapter = new SqlDataAdapter(sqlCommand);
            DataSet dataSet = new DataSet();

            // Veriler DataSet'e dolduruluyor.
            sqlDataAdapter.Fill(dataSet);

            // DataGrid'e arama sonuçları atanıyor.
            dataGridArac.DataSource = dataSet.Tables[0];
        }
        catch (Exception hata)
        {
            // Hata oluşursa kullanıcıya hata mesajı gösteriliyor.
            MessageBox.Show("Hata Meydana Geldi: " + hata.Message, "Mesaj Kutusu",
              MessageBoxButtons.OKCancel, MessageBoxIcon.Error);
        }
    }
}
```

C# İLE TEMEL CRUD İŞLEMLERİ

- **KAYIT GÜNCELLEME**
- veri tabanındaki bir tablo üzerinde güncelleme yapabilmek için “Update” deyimi kullanılmaktadır.
- DataGridView üzerinde yer alan bir kaydı veri tabanı üzerinde güncellemek için iki (2) adımlı bir işlem gerçekleştirilmelidir.
- Birinci adımda DataGridView üzerinde seçilen kayda ait alanlar veri giriş formu üzerinde ilgili alanlara aktarılır.
- “dataGridArac” isimli kontrolün “SelectionChanged” olayına yazılacak yardımıyla DataGridView üzerinde seçilen satırda yer alan bilgiler veri giriş formuna aktarılır.

C# İLE TEMEL CRUD İŞLEMLERİ

- **KAYIT GÜNCELLEME**

- Birinci adımda **“dataGridArac”** kontrolünün **“SelectionChanged””** olayını kodluyoruz.

```
private void dataGridArac_SelectionChanged(object sender, EventArgs e)
{
    txtMarka.Text = dataGridArac.CurrentRow.Cells["Marka"].Value.ToString();
    txtModel.Text = dataGridArac.CurrentRow.Cells["Model"].Value.ToString();
    txtYil.Text = dataGridArac.CurrentRow.Cells["Yil"].Value.ToString();
    txtRenk.Text = dataGridArac.CurrentRow.Cells["RenkAdi"].Value.ToString();
}
```

C# İLE TEMEL CRUD İŞLEMLERİ

- **KAYIT GÜNCELLEME**
- İkinci adımda **“btnGuncelle”** kontrolünün **“Click”** olayına kodluyoruz.

C# İLE TEMEL CRUD İŞLEMLERİ

- **KAYIT GÜNCELLEME**

```
private void btnGuncelle_Click(object sender, EventArgs e)
{
    // SqlConnection ile veritabanına bağlantı oluşturuluyor; using ifadesi ile otomatik kapama
      sağlanıyor.
    using (SqlConnection sqlConnection = new SqlConnection(conString))
    {
        try
        {
            // SQL güncelleme komutu tanımlanıyor.
            SqlCommand sqlCommand = new SqlCommand("UPDATE Arac SET Marka = @Marka, Model = @Model, Yil =
              @Yil, RenkAdi = @Renk WHERE AracId = @AracId", sqlConnection);

            // Parametreler ekleniyor; kullanıcıdan alınan veriler SQL sorgusuna dahil ediliyor.
            sqlCommand.Parameters.AddWithValue("@Marka", txtMarka.Text);
            sqlCommand.Parameters.AddWithValue("@Model", txtModel.Text);
            sqlCommand.Parameters.AddWithValue("@Yil", txtYil.Text);
            sqlCommand.Parameters.AddWithValue("@Renk", txtRenk.Text);
            sqlCommand.Parameters.AddWithValue("@AracId", dataGridArac.CurrentRow.Cells
              ["AracId"].Value); // AracId, seçilen satırdan alınıyor.
```

C# İLE TEMEL CRUD İŞLEMLERİ

- **KAYIT GÜNCELLEME**

```
            // Bağlantıyı aç ve komutu çalıştır.
            sqlConnection.Open();
            sqlCommand.ExecuteNonQuery();

            // Güncelleme başarılı olduğunda kullanıcıya mesaj gösteriliyor.
            MessageBox.Show("Kayıt Güncellendi.", "Mesaj Kutusu", MessageBoxButtons.OK,
              MessageBoxIcon.Information);
        }
        catch (Exception hata)
        {
            // Hata oluşursa kullanıcıya hata mesajı gösteriliyor.
            MessageBox.Show("Hata Meydana Geldi: " + hata.Message, "Mesaj Kutusu",
              MessageBoxButtons.OKCancel, MessageBoxIcon.Error);
        }
        // using bloğu sayesinde bağlantı otomatik olarak kapatılır.
    }
}
```

C# İLE TEMEL CRUD İŞLEMLERİ

- **KAYIT SİLME**
- Veri tabanındaki bir tablodan kayıt silmek için **“Delete”** deyimi kullanılmaktadır.
- **“btnSil”** kontrolünün **“Click”** olayı:

C# İLE TEMEL CRUD İŞLEMLERİ

- **KAYIT SİLME**

```
private void btnSil_Click(object sender, EventArgs e)
{
    // SqlConnection ile veritabanına bağlantı oluşturuluyor; using ifadesi ile otomatik kapama
      sağlanıyor.
    using (SqlConnection sqlConnection = new SqlConnection(conString))
    {
        try
        {
            // SQL silme komutu tanımlanıyor.
            SqlCommand sqlCommand = new SqlCommand("DELETE FROM Arac WHERE AracId = @AracId",
              sqlConnection);

            // Seçilen satırdan AracId değeri alınıyor.
            sqlCommand.Parameters.AddWithValue("@AracId", dataGridArac.CurrentRow.Cells["AracId"].Value);

            // Bağlantıyı aç ve komutu çalıştır.
            sqlConnection.Open();
            sqlCommand.ExecuteNonQuery();
```

C# İLE TEMEL CRUD İŞLEMLERİ

- **KAYIT SİLME**

```
            // Silme işlemi başarılı olduğunda kullanıcıya mesaj gösteriliyor.
            MessageBox.Show("Kayıt Silindi.", "Mesaj Kutusu", MessageBoxButtons.OK,
              MessageBoxIcon.Information);
        }
        catch (Exception hata)
        {
            // Hata oluşursa kullanıcıya hata mesajı gösteriliyor.
            MessageBox.Show("Hata Meydana Geldi: " + hata.Message, "Mesaj Kutusu",
              MessageBoxButtons.OKCancel, MessageBoxIcon.Error);
        }
        // using bloğu sayesinde bağlantı otomatik olarak kapatılır.
    }
}
```